

MODULE 1 - Introduction to Software Testing

Definition Of Software Testing

- Testing is the **execution** of programs **with the intent of finding defects**.
- Testing is a the **process of exercising a software component**
 - using a **selected set of test cases**,
 - with the intent of **revealing defects** and **evaluating quality**.
- Software testing is a **technical investigation** of a product, i.e. **an empirical search** for **quality-related information** of **value** to a project's stakeholders

Bug, Fault & Failure

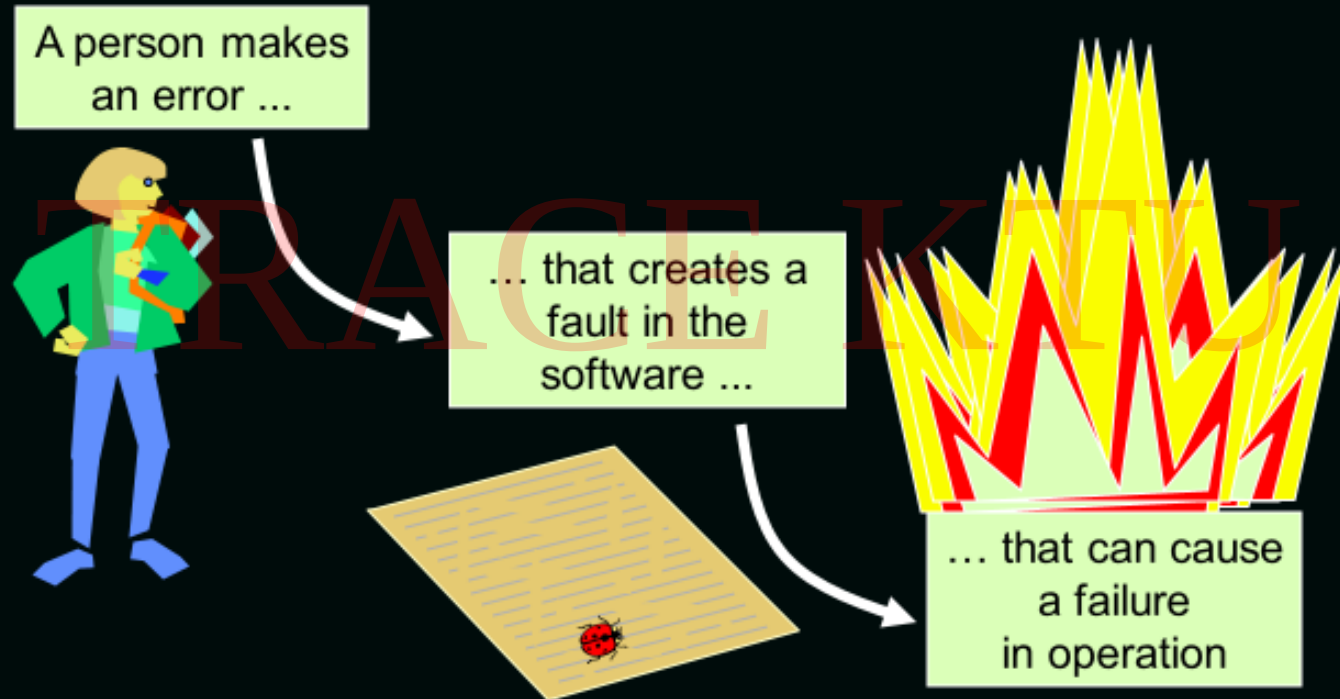
- A person makes an **Error**
 - That creates a **fault** in software
 - That can cause a **failure** in operation
- **Error** : An error is a human action that produces the incorrect result that results in a fault.
- **Bug** : The presence of error at the time of execution of the software.
- **Fault** : State of software caused by an error.
- **Failure** : Deviation of the software from its expected result. It is an event.
- **Defect** : A defect is an error or a bug, in the application which is created. A programmer while designing and building the software can make mistakes or error. These mistakes or errors mean that there are flaws in the software. These are called defects.

What is a “bug”?

- **Error:** a human action that produces an incorrect result
- **Fault:** a manifestation of an error in software
 - also known as a defect or bug
 - if executed, a fault may cause a failure
- **Failure:** deviation of the software from its expected delivery or service
 - (found defect)

Failure is an event; fault is a state of the software, caused by an error

Error - Fault - Failure



Why do defects occur in software?

Software is written by human beings

- 🕒 Who know something, but not everything
- 🕒 Who have skills, but aren't perfect
- 🕒 Who don't usually use rigorous methods
- 🕒 Who do make mistakes (errors)

Under increasing pressure to deliver to strict deadlines

- 🕒 No time to check, assumptions may be wrong
- 🕒 Systems may be incomplete

Software is complex, abstract and invisible

- 🕒 Hard to understand
- 🕒 Hard to see if it is complete or working correctly
- 🕒 No one person can fully understand large systems
- 🕒 Numerous external interfaces and dependencies

Sources of defects

Education



Developers does not understand well enough what he or she is doing



Lack of proper education leads to errors in specification, design, coding, and testing

Communication



Developers do not know enough



Information does not reach all stakeholders



Information is lost

Oversight



Omitting to do necessary things

Transcription



Developer knows what to do but simply makes a mistake

Process



Process is not applicable for the actual situation



Process places restrictions that cause errors

Objective of testing

- To find defects before they cause a production system to fail.
- To bring the tested software, after correction of the identified defects and retesting, to an acceptable level of quality.
- To perform the required tests efficiently and effectively, within the limits budgetary and scheduling limitation.
- To compile a record of software errors for use in error prevention (by corrective and preventive actions)

Ariane-5 (1996)

- European Space Agency (ESA) launched this satellite
- Thirty seconds into liftoff it lost guidance and altitude information
- Veered completely off-course and automatically self-destructed in 40 seconds
- Caused by inertial reference system converting a 64-bit float to a 16-bit signed integer which produced an overflow error
- Cost: \$254 million



Therac-25 Medical Accelerator Disaster (1985)

- Therac-25 was a radiation therapy machine used in the treatment of cancer patients
- Two modes: precision electron beam and a megavolt X-ray mode which required shielding, filters, and an ion chamber to keep beams safely on target
- Some patients exposed to megavolt X-rays on accident
- Caused by race condition in the software
- Cost: 6 radiation overdoses



Pentium Floating-Point Miscalculations (1994)

- Intel launched Pentium processor in March 1993
- Intel implemented lookup tables for floating points
- Some table entries did not make it onto the chip
- 1-in-9 billion chance of a miscalculation
- Intel discovered bug in June 1994
- In October 1994, math professor Thomas Nicely discovered miscalculations with floating points
- Cost: \$759.4 million



Software Quality Concept

Quality : Quality means consistently meeting customer needs in terms of requirement, Cost and delivery schedule.

Quality of s/w is reasonably bug free, delivered on time and within budget, meets requirements and exceptions and is maintainable

Software Quality Concept

- **Quality Control:** “All **defined** work products and **Measurable Specifications**” are compared with the output of each process.
- Quality control focuses on operational technique and activities used to fulfill and verify requirement of quality.
- - S/w quality involves – Series of inspection, reviews, and test used through out the s/w process.

Quality Assurance (QA): consist of Auditing and reporting procedure. Which are used to provide necessary data to management. In order to make decision.

- Goal of Quality Assurance is to provide adequate confidence that a product or services is of the and quality expected by the customer.
- If the data provided through quality assurance identify problems, then it is management's responsibility to address the problems & apply the necessary resources to resolve quality issues.

Level 0 Thinking

- n Testing is the **same** as debugging .
- n Does not distinguish between incorrect **behavior** and mistakes in the program
- n Does not help develop software that is reliable or safe

Level 1 Thinking

- n Purpose is to show correctness
- n Correctness is impossible to achieve
- n What do we know if no failures?
 - Good software or bad tests?
- n Test engineers have no:
 - Strict goal
 - Real stopping rule
 - Formal test technique
 - Test managers are powerless

Level 2 Thinking

- n Purpose is to show failures
- n Looking for failures is a negative activity
- n Puts testers and developers into an adversarial relationship
- n What if there are no failures?

Level 3 Thinking

- n Testing can only show the presence of failures
- n Whenever we use software, we incur some risk
- n Risk may be small and consequences unimportant
- n Risk may be great and consequences catastrophic
- n Testers and developers cooperate to reduce risk

Level 4 Thinking

A mental discipline that increases quality

- n Testing is only **one way** to increase quality
- n Test engineers can become **technical leaders** of the project
- n Primary responsibility to **measure and improve** software quality
- n Their expertise should **help the developers**

What is Verification?

- **Definition** : *The process of evaluating software to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase.*
- Verification is a static practice of verifying documents, design, code and program. It includes all the activities associated with producing high quality software: inspection, design analysis and specification analysis. It is a relatively objective process.
- Verification will help to determine whether the software is of high quality, but it will not ensure that the system is useful. Verification is concerned with whether the system is well-engineered and error-free.
- **Methods of Verification** : Static Testing
 - *Walkthrough*
 - *Inspection*
 - *Review*

What is Validation?

- **Definition:** *The process of evaluating software during or at the end of the development process to determine whether it satisfies specified requirements.*
- Validation is the process of evaluating the final product to check whether the software meets the customer expectations and requirements. It is a dynamic mechanism of validating and testing the actual product.
- **Methods of Validation :** [Dynamic Testing](#)
- *Testing*
- *End Users*

Verification

1. **Verification** is a static practice of verifying documents, design, code and program.
2. It does not involve executing the code.
3. It is human based checking of documents and files.
4. **Verification** uses methods like inspections, reviews, walkthroughs, and Desk-checking etc.
5. **Verification** is to check whether the software conforms to specifications.
6. It can catch errors that validation cannot catch. It is low level exercise.
7. Target is requirements specification, application and software architecture, high level, complete design, and database design etc.
8. **Verification** is done by QA team to ensure that the software is as per the specifications in the SRS document.
9. It generally comes first-done before validation.

Validation

1. **Validation** is a dynamic mechanism of validating and testing the actual product.
2. It always involves executing the code.
3. It is computer based execution of program.
4. **Validation** uses methods like black box (functional) testing, gray box testing, and white box (structural) testing etc.
5. **Validation** is to check whether software meets the customer expectations and requirements.
6. *It can catch errors that verification cannot catch. It is High Level Exercise.*
7. Target is actual product-a unit, a module, a bent of integrated modules, and effective final product.
8. **Validation** is carried out with the involvement of testing team.
9. It generally follows after **verification**.

General meaning of test case concept

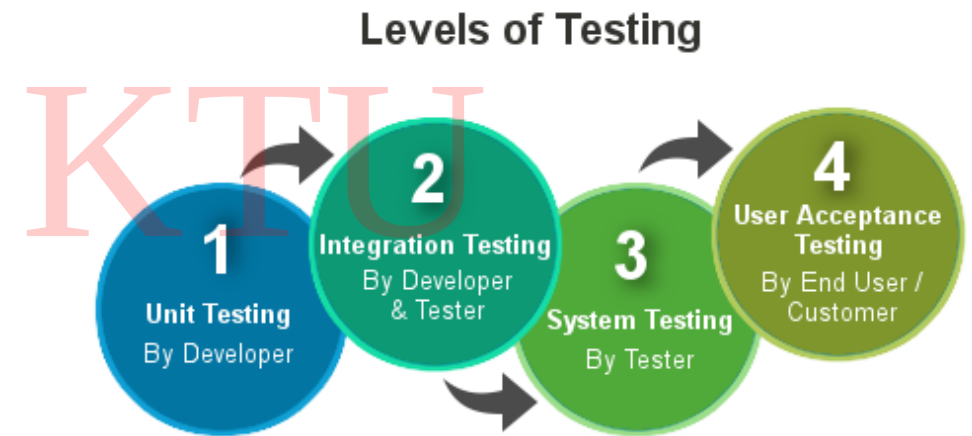
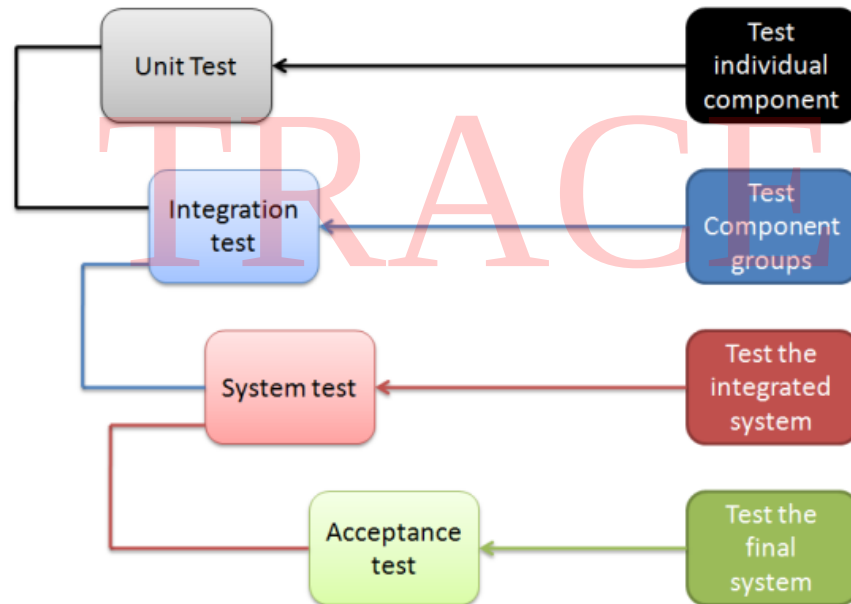
- A test case is a set of actions with the expected results based on requirements for the system
- Test cases have to provide full coverage of all technical (system) requirements for the product which is being tested
- Test cases should include a description of the functionality to be tested, and the preparation required to ensure that the test can be conducted.
- An effective test cases have to be developed from the position of “Constructive demolition”

Coverage Criteria

Coverage criteria are **adequacy measures to qualify if a test objective is satisfied when executing test cases on a system under test.**

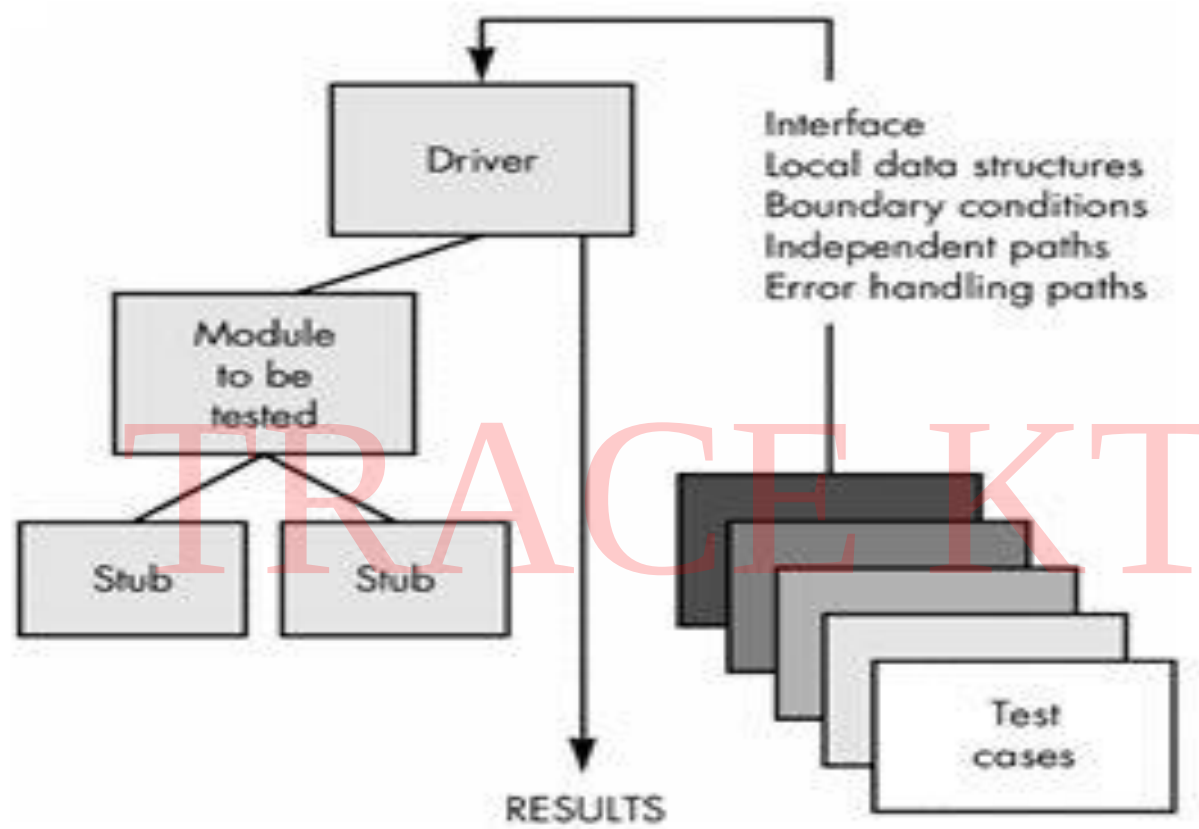
Coverage criteria are established to estimate the quality of test cases, and criteria combinations are considered in software testing.

Levels Of Testing



Unit Testing

- **Unit Testing** is a level of software testing where **individual units/components** of a software are tested. The purpose is to **validate that each unit of the software performs as designed**.
- Unit Testing is the first level of testing and is performed prior to Integration Testing.
- A unit is the smallest testable part of software. It usually has one or a few inputs and usually a single output.
- It is executed by the Developer.
- Unit Testing is performed by using the White Box Testing method
- Example: - A function, method, Loop or statement in program is working fine.



Drivers

- Drivers are used in bottom-up integration testing approach.
- It can simulate the behavior of upper-level module that is not integrated yet.
- Drivers modules act as the temporary replacement of module and act as the actual products.
- Drivers are also used for interact with external system and usually complex than stubs.
- **Driver:** Calls the Module to be tested.

Now suppose you have modules B and C ready but module A which calls functions from module B and C is not ready so developer will write a dummy piece of code for module A which will return values to module B and C. This dummy piece of code is known as driver.

Stubs

- Stubs are used in top down integration testing.
- It can simulate the behavior of lower-level module that are not integrated.
- They act as a temporary replacement of module and provide same output as actual product.
- When needs to interact with external system then also stubs are used.
- **Stub:** Is called by the Module under Test.

Assume you have 3 modules, Module A, Module B and module C. Module A is ready and we need to test it, but module A calls functions from Module B and C which are not ready, so developer will write a dummy module which simulates B and C and returns values to module A. This dummy module code is known as stub.

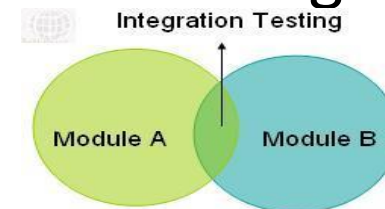
	Stub	Driver
Type	Dummy codes	Dummy codes
Description	Routines that don't actually do anything except declare themselves and the parameters they accept. The rest of the code can then take these parameters and use them as inputs	Routines that don't actually do anything except declare themselves and the parameters they accept. The rest of the code can then take these parameters and use them as inputs
Used in	Top Down Integration	Bottom Up Integration
Purpose	To allow testing of the upper levels of the code, when the lower levels of the code are not yet developed.	To allow testing of the lower levels of the code, when the upper levels of the code are not yet developed.

Benefits

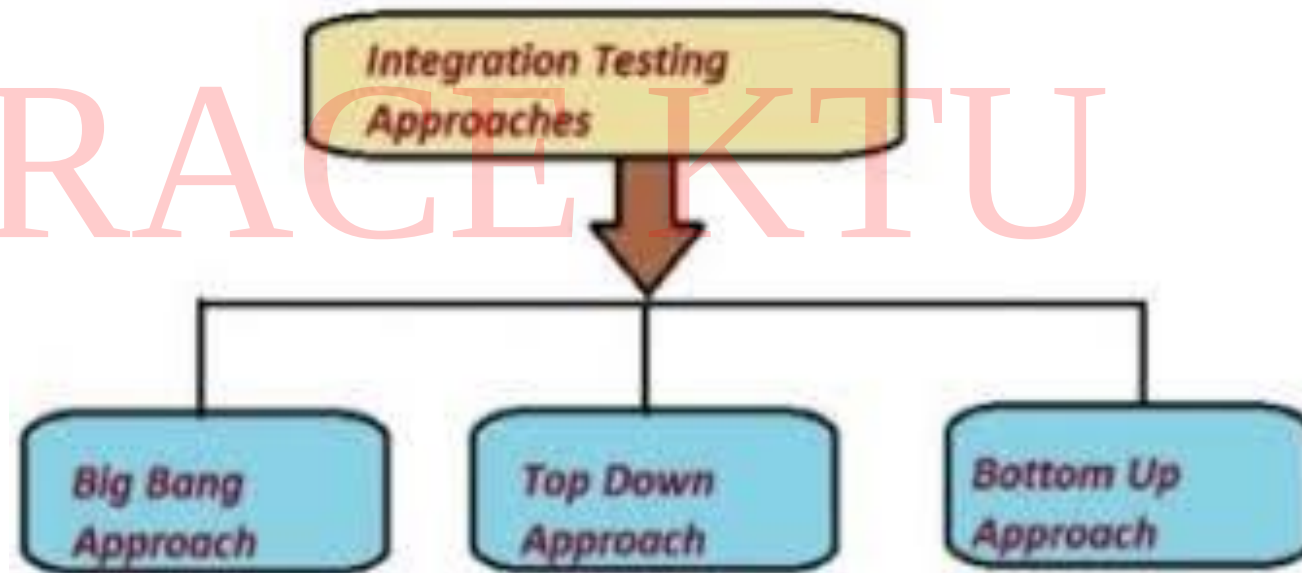
- Unit testing increases confidence in changing/ maintaining code. If good unit tests are written and if they are run every time any code is changed, we will be able to promptly catch any defects introduced due to the change.
- Codes are more reusable.
- Development is faster.
- The cost of fixing a defect detected during unit testing is lesser in comparison to that of defects detected at higher levels.
- Debugging is easy.

Integration Testing

- **Integration Testing** is a [level of software testing](#) where individual units are combined and tested as a group.
- In integration Testing, individual software modules are integrated logically and tested as a group.
- **Integration testing** tests integration or interfaces between components, interactions to different parts of the system such as an operating system, file system and hardware or interfaces between systems.
- As displayed in the image below when two different modules 'Module A' and 'Module B' are integrated then the integration testing is done.

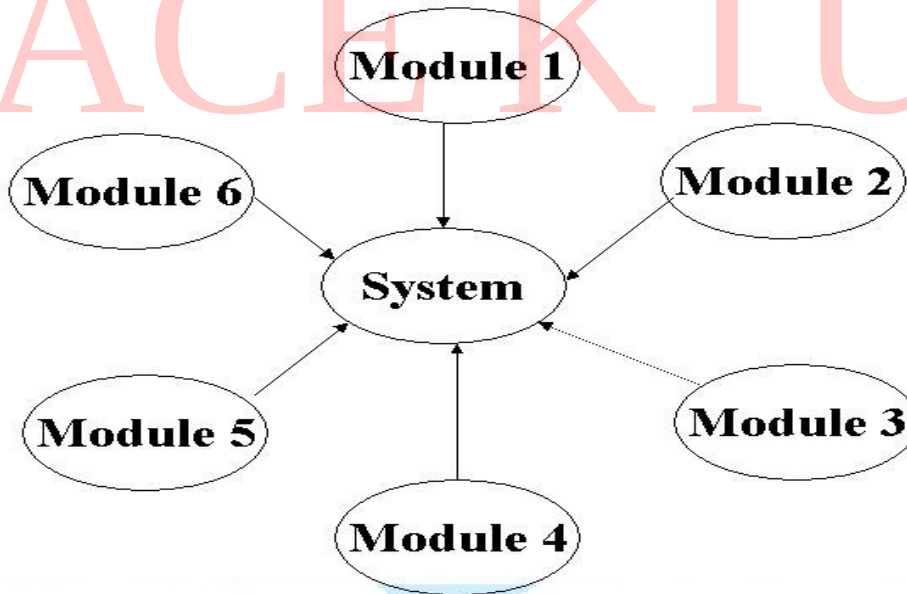


Integration Testing Approaches



Big Bang integration testing

- In Big Bang integration testing all components or modules are integrated simultaneously, after which everything is tested as a whole. As per the below image all the modules from 'Module 1' to 'Module 6' are integrated simultaneously.



Pros And Cons

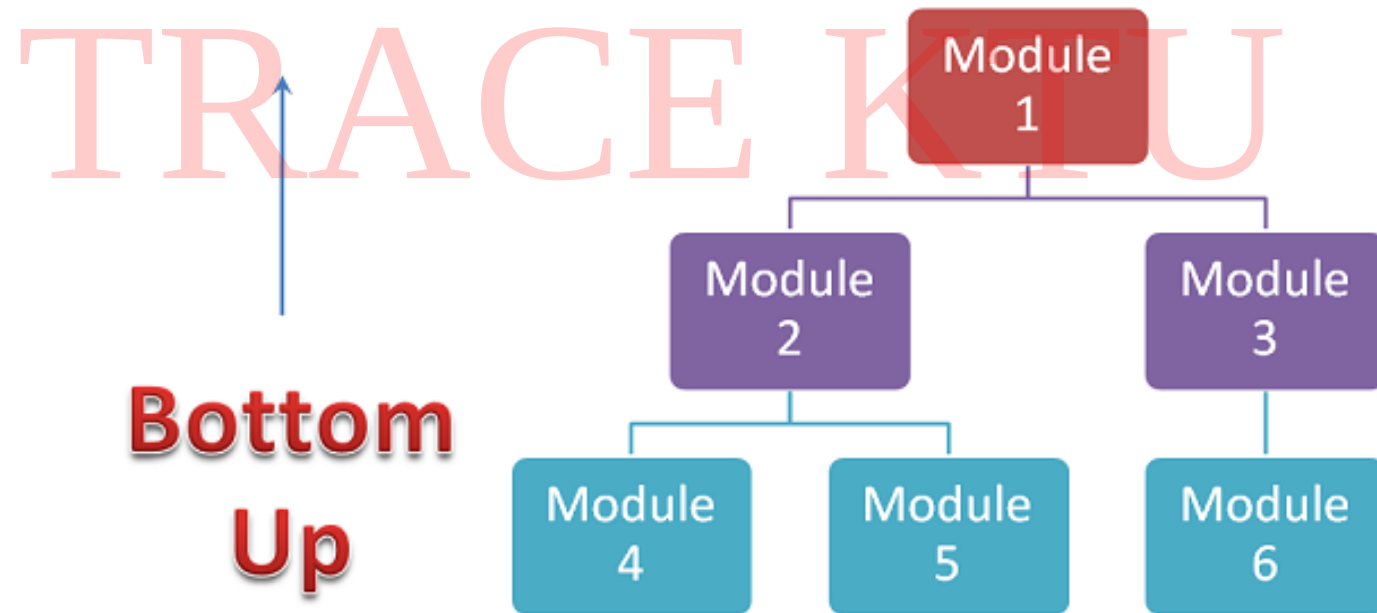
- Convenient for small systems.
- Fault Localization is difficult.
- Since all modules are tested at once, high risk critical modules are not isolated and tested on priority.
- Since the integration testing can commence only after "all" the modules are designed, testing team will have less time for execution in the testing phase.

Incremental Approach:

- In this approach, testing is done by joining two or more modules that are ***logically related***. Then the other related modules are added and tested for the proper functioning. Process continues until all of the modules are joined and tested successfully.
- This process is carried out by using dummy programs called **Stubs and Drivers**. Stubs and Drivers do not implement the entire programming logic of the software module but just simulate data communication with the calling module.
- **Stub**: Is called by the Module under Test.
- **Driver**: Calls the Module to be tested.

Bottom up Integration

- In the bottom up strategy, each module at lower levels is tested with higher modules until all modules are tested.
- It takes help of Drivers for testing



Advantages:

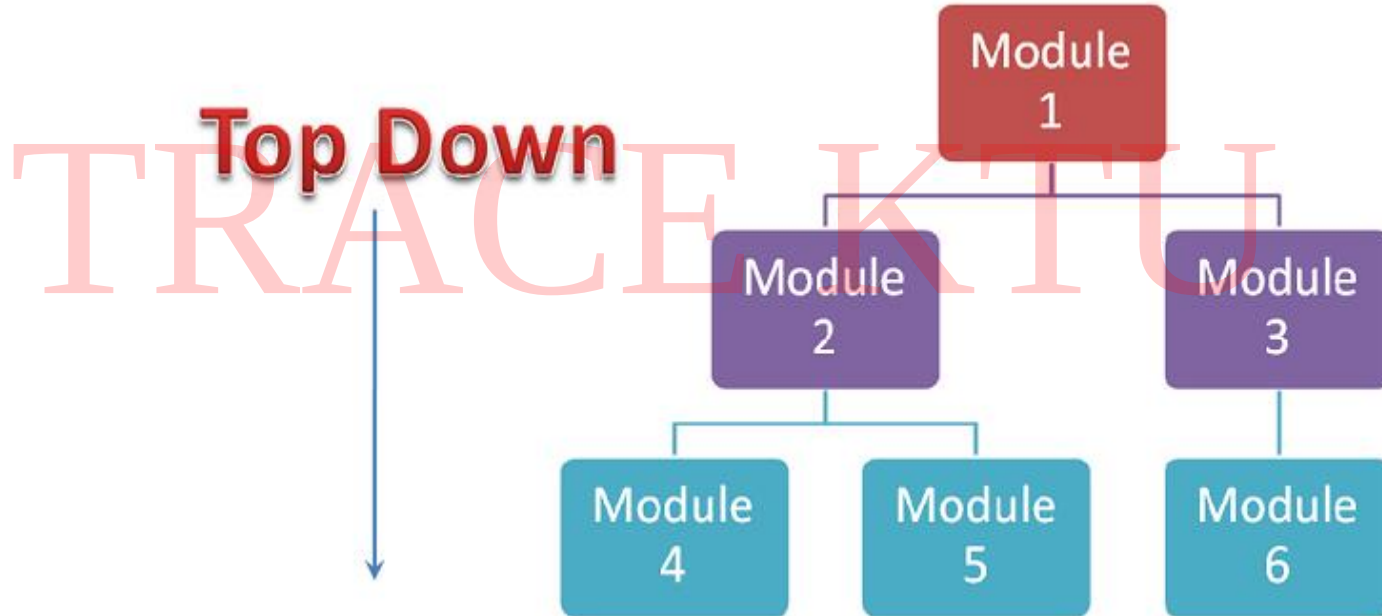
- Fault localization is easier.
- No time is wasted waiting for all modules to be developed unlike Big-bang approach

Disadvantages:

- Critical modules (at the top level of software architecture) which control the flow of application are tested last and may be prone to defects.
- Early prototype is not possible

Top down Integration:

- In Top to down approach, testing takes place from top to down following the control flow of the software system.
- Takes help of stubs for testing.



Advantages:

- Fault Localization is easier.
- Possibility to obtain an early prototype.
- Critical Modules are tested on priority; major design flaws could be found and fixed first.

Disadvantages:

- Needs many Stubs.
- Modules at lower level are tested inadequately.

Unit Testing

Unit testing is a type of testing to check if the small piece of code is doing what it is suppose to do.

Unit testing checks a single component of an application.

The scope of Unit testing is narrow, it covers the Unit or small piece of code under test.

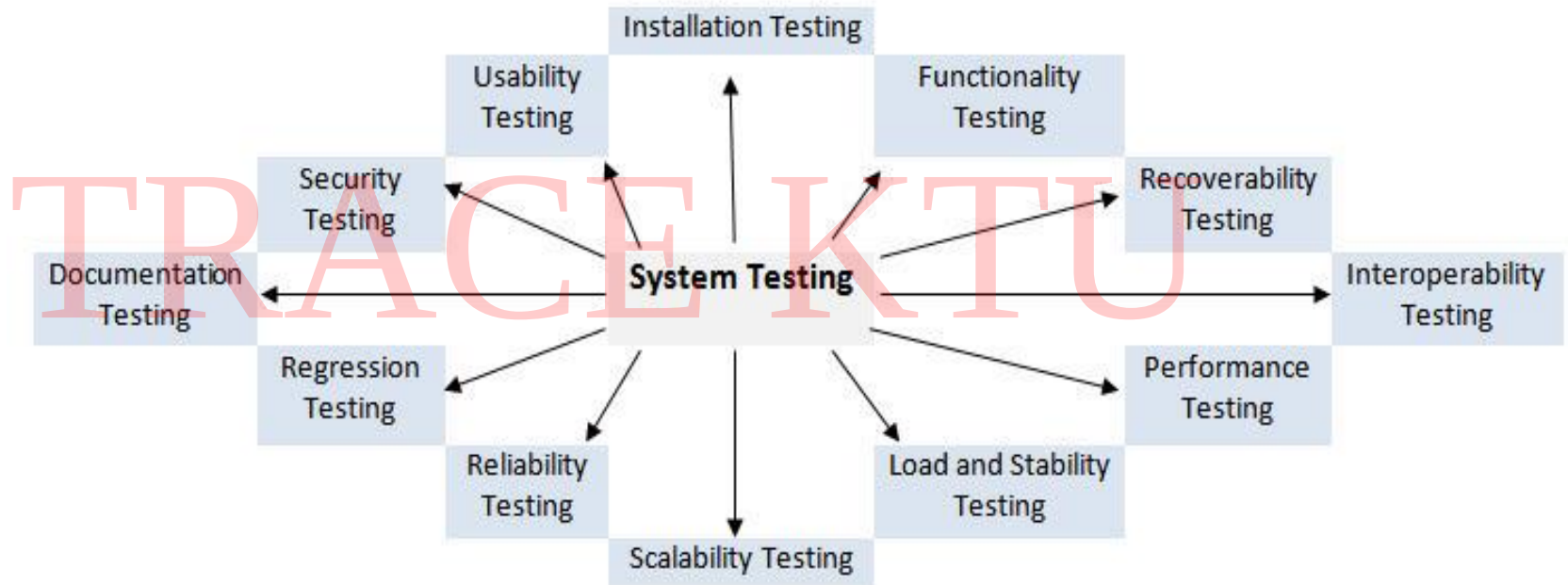
Integration Testing

Integration testing is a type of testing to check if different pieces of the modules are working together.

The behavior of integration modules is considered in the Integration testing.

The scope of Integration testing is wide, it covers the whole application under test and it

System Testing



System Testing - © www.SoftwareTestingHelp.com

- The process of testing of an integrated hardware and software system to verify that the system meets its specified requirements.
- It is performed when integration testing is completed.
- It is mainly a black box type testing. This testing evaluates working of the system from a user point of view, with the help of specification document. It does not require any internal knowledge of system like design or structure of the code.
- It contains functional and non-functional areas of application/product.
- System testing is performed in the context of a System Requirement Specification (SRS) and/or a Functional Requirement Specifications (FRS). It is the final test to verify that the product to be delivered meets the specifications mentioned in the requirement document. It should investigate both functional and non-functional requirements.

- It mainly focuses on following:
 - External interfaces
 - complex functionalities
 - Security
 - Recovery
 - Performance
 - Operator and user's smooth interaction with system
 - Documentation
 - Usability
 - Load / Stress

Performance Testing

- Performance Testing is a type of testing to ensure software applications will perform well under their expected workload.
- A software application's performance like its response time, reliability, resource usage and scalability do matter.
- The goal of Performance Testing is not to find bugs but to eliminate performance bottlenecks.
- The focus of Performance Testing is checking a software program's
 - Speed - Determines whether the application responds quickly
 - Scalability - Determines maximum user load the software application can handle.
 - Stability - Determines if the application is stable under varying loads

Performance Testing Process



1) Identify your testing environment –

Do proper requirement study & analyzing test goals and its objectives. Also determine the testing scope along with test Initiation Checklist. Identify the logical and physical production architecture for performance testing, identify the software, hardware and networks configurations required for kick off the performance testing. Compare the both test and production environments while identifying the testing environment. Get resolve the environment-related concerns if any, analyze that whether additional tools are required for performance testing. This step also helps to identify the probable challenges tester may face while performance testing.

2) Identify the performance acceptance criteria –

Identify the desired performance characteristics of the application like Response time, Throughput and Resource utilization.

3) Plan & design performance tests –

Planning and designing performance tests involves identifying key usage scenarios, determining appropriate variability across users, identifying and generating test data, and specifying the metrics to be collected. Ultimately, these items will provide the foundation for workloads and workload profiles. The output of this stage is prerequisites for Test execution are ready, all required resources, tools & test data are ready.

4) Configuring the test environment –

Prepare with conceptual strategy, available tools, designed tests along with testing environment before execution. The output of this stage is configured load-generation environment and resource-monitoring tools.

5) Implement test design –

According to test planning and design create your performance tests.

6) Execute the tests –

Collect and analyze the data.

Problem Investigation like bottlenecks (memory, disk, processor, process, cache, network, etc.) resource usage like (memory, CPU, network, etc.,)

Generate the Performance analysis reports containing all performance attributes of the application.

Based on the analysis prepare recommendation report.

Repeat the above test for the new build received from client after fixing the bugs and implementing the recommendations

7) Analyze Results, Report, and Retest

Consolidate, analyze and share test results.

Based on the test report re-prioritize the test & re-execute the same. If any specific test result within the specified metric limit & all results are between the thresholds limits then testing of same scenario on particular configuration is completed.

Test objectives frequently include the following:

- **Response time.** For example, the product catalog must be displayed in less than 3 seconds.
- **Throughput.** For example, the system must support 100 transactions per second.
- **Resource utilization.** A frequently overlooked aspect is the amount of resources your application is consuming, in terms of processor, memory, disk input output (I/O), and network I/O.

Types of Performance Testing

- Load test
- Stress test
- Data Volume Testing
- Storage Testing

Stress Testing

- Stress Testing is performance testing type to check the stability of software when hardware resources are not sufficient like CPU, memory, disk space etc.
- It is performed to find the upper limit capacity of the system and also to determine how the system performs if the current load goes well above the expected maximum.
- Main parameters to focus during Stress testing are “Response Time” and “Throughput”.
- Stress testing is Negative testing where we load the software with large number of concurrent users/processes which cannot be handled by the systems hardware resources. This testing is also known as **Fatigue testing**

Usability Testing

- In **usability testing** basically the testers tests the ease with which the user interfaces can be used.
- It tests that whether the application or the product built is user-friendly or not.
- Usability Testing is a black box testing technique.
- Usability testing also reveals whether users feel comfortable with your application or Web site according to different parameters – the flow, navigation and layout, speed and content – especially in comparison to prior or similar applications.
- Usability Testing tests the following features of the software.
 - — How easy it is to use the software.
 - — How easy it is to learn the software.
 - — How convenient is the software to end user.

Usability testing includes the following five components:

- **Learnability:** How easy is it for users to accomplish basic tasks the first time they encounter the design?
- **Efficiency:** How fast can experienced users accomplish tasks?
- **Memorability:** When users return to the design after a period of not using it, does the user remember enough to use it effectively the next time, or does the user have to start over again learning everything?
- **Errors:** How many errors do users make, how severe are these errors and how easily can they recover from the errors?
- **Satisfaction:** How much does the user like using the system?

Benefits of usability testing to the end user or the customer:

- Better quality software
- Software is easier to use
- Software is more readily accepted by users
- Shortens the learning curve for new users

TRACE KTU

Acceptance Testing

- **Acceptance Testing** is a [level of the software testing](#) where a system is tested for acceptability.
- The purpose of this test is to evaluate the system's compliance with the business requirements and assess whether it is acceptable for delivery.
- Usually, [Black Box Testing](#) method is used in Acceptance Testing.
- Acceptance Testing is performed after [System Testing](#) and before making the system available for actual use.
- The acceptance test cases are executed against the test data or using an acceptance test script and then the results are compared with the expected ones.
- The goal of acceptance [testing](#) is to establish confidence in the system.
- Acceptance testing is most often focused on a validation type testing.

Acceptance Criteria

Acceptance criteria are defined on the basis of the following attributes

- Functional Correctness and Completeness
- Data Integrity
- Data Conversion
- Usability
- Performance
- Timeliness
- Confidentiality and Availability
- Installability and Upgradability
- Scalability
- Documentation

Types Of Acceptance Testing

- User Acceptance test
- Operational Acceptance test
- Contract Acceptance testing
- Compliance acceptance testing

TRACE KTU

- **User Acceptance test**

It focuses mainly on the functionality thereby validating the fitness-for-use of the system by the business user. The user acceptance test is performed by the users and application managers.

- **Operational Acceptance test**

Also known as Production acceptance test validates whether the system meets the requirements for operation. In most of the organization the operational acceptance test is performed by the system administration before the system is released. The operational acceptance test may include testing of backup/restore, disaster recovery, maintenance tasks and periodic check of security vulnerabilities.

- **Contract Acceptance testing**

It is performed against the contract's acceptance criteria for producing custom developed software. Acceptance should be formally defined when the contract is agreed.

- **Compliance acceptance testing**

It is also known as regulation acceptance testing is performed against the regulations which must be adhered to, such as governmental, legal or safety regulations.

Advantages Of Acceptance Testing

- The functions and features to be tested are known.
- The details of the tests are known and can be measured.
- The tests can be automated, which permits regression testing.
- The progress of the tests can be measured and monitored.
- The acceptability criteria are known.

Disadvantages Of Acceptance Testing

- Requires significant resources and planning.
- The tests may be a re-implementation of system tests.
- It may not uncover subjective defects in the software, since you are only looking for defects you expect to find.

Beta Testing

- **Beta Testing** is also known as field testing. It takes place at **customer's site**. It sends the system/software to users who install it and use it under real-world working conditions.
- A beta test is the second phase of [software testing](#) in which a sampling of the intended audience tries the product out
- The goal of beta testing is to place your application in the hands of real users outside of your own engineering team to discover any flaws or issues from the user's perspective that you would not want to have in your final, released version of the application.
- Beta testing can be considered “pre-release testing.

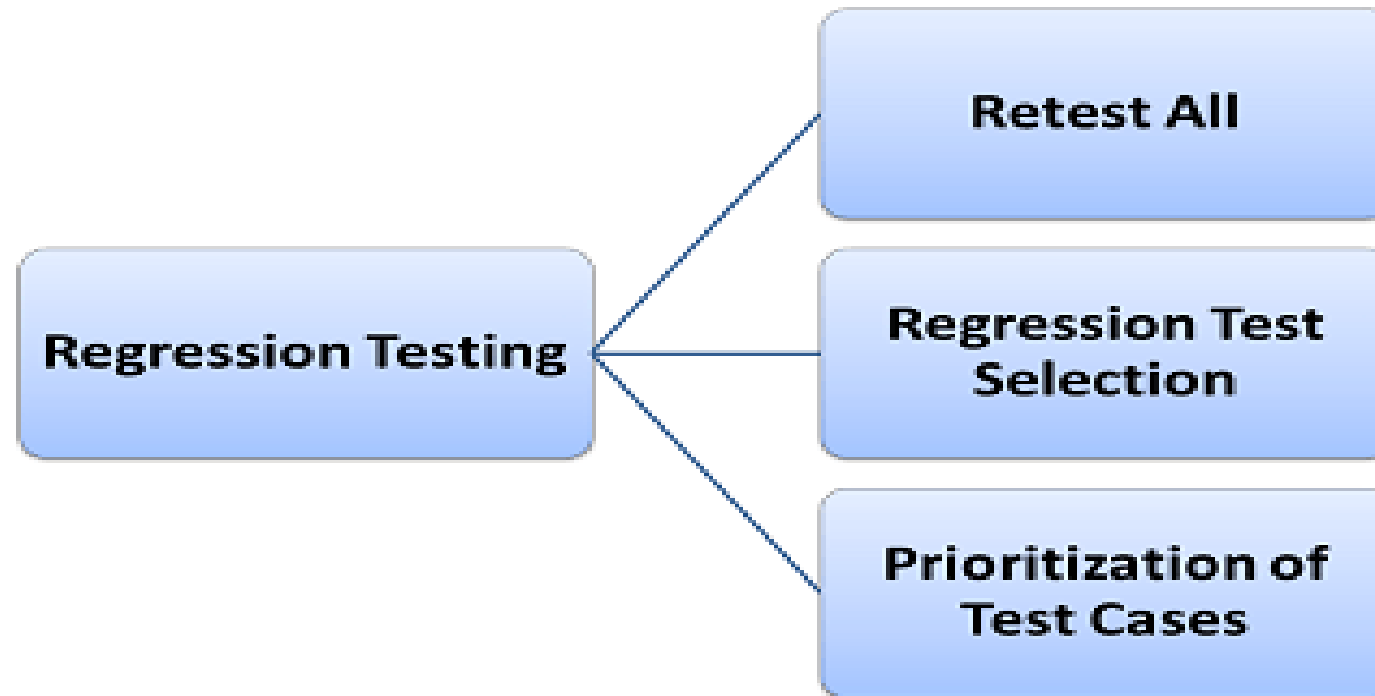
Advantages of beta testing

- You have the opportunity to get your application into the hands of users prior to releasing it to the general public.
- Users can install, test your application, and send feedback to you during this beta testing period.
- Your beta testers can discover issues with your application that you may have not noticed, such as confusing application flow, and even crashes.
- Using the feedback you get from these users, you can fix problems before it is released to the general public.
- The more issues you fix that solve real user problems, the higher the quality of your application when you release it to the general public.
- Having a higher-quality application when you release to the general public will increase customer satisfaction.
- These users, who are early adopters of your application, will generate excitement about your application.

Regression Testing

- Regression Testing is defined as a type of software testing to confirm that a recent program or code change has not adversely affected existing features.
- Regression Testing is nothing but full or partial selection of already executed test cases which are re-executed to ensure existing functionalities work fine.
- This testing is done to make sure that new code changes should not have side effects on the existing functionalities. It ensures that old code still works once the new code changes are done.
- Regression Testing is required when there is a
 - ✓ Change in requirements and code is modified according to the requirement
 - ✓ New feature is added to the software
 - ✓ Defect fixing
 - ✓ Performance issue fix

Regression Testing Techniques :



Retest All

This is one of the methods for Regression Testing in which all the tests in the existing test bucket or suite should be re-executed. This is very expensive as it requires huge time and resources.

Regression Test Selection

Instead of re-executing the entire test suite, it is better to select part of test suite to be run

Test cases selected can be categorized as 1) Reusable Test Cases 2) Obsolete Test Cases.

Re-usable Test cases can be used in succeeding regression cycles.

Obsolete Test Cases can't be used in succeeding cycles.

Prioritization of Test Cases

Prioritize the test cases depending on business impact, critical & frequently used functionalities. Selection of test cases based on priority will greatly reduce the regression test suite.

Selecting test cases for regression testing

It was found from industry data that good number of the defects reported by customers were due to last minute bug fixes creating side effects and hence selecting the [Test Case](#) for regression testing is an art and not that easy.

Effective Regression Tests can be done by selecting following test cases –

- ✓ Test cases which have frequent defects
- ✓ Functionalities which are more visible to the users
- ✓ Test cases which verify core features of the product
- ✓ Test cases of Functionalities which has undergone more and recent changes
- ✓ All Integration Test Cases
- ✓ All Complex Test Cases
- ✓ Boundary value test cases
- ✓ Sample of Successful test cases
- ✓ Sample of Failure test cases

Regression Testing Tools

If your software undergoes frequent changes, regression testing costs will escalate. In such cases, Manual execution of test cases increases test execution time as well as costs.

Following are most important tools used for both functional and regression testing:

Selenium: This is an open source tool used for automating web applications. Selenium can be used for browser based regression testing.

Quick Test Professional (QTP): HP Quick Test Professional is automated software designed to automate functional and regression test cases. It uses VBScript language for automation. It is a Data driven, Keyword based tool.

Rational Functional Tester (RFT): IBM's rational functional tester is a Java tool used to automate the test cases of software applications. This is primarily used for automating regression test cases and it also integrates with Rational Test Manager.

What is Functional Testing?

Functional testing is a type of software testing in which software is tested against functional requirements/specifications with the intent to find a bug/defect(s) if any.

Functional testing is different from system testing as functional testing verifies a program by checking it against the design document while system testing validates a program by checking it against the published user or system requirement.

Functional testing consist of six steps-

1. The identification of functions that the software is expected to perform
2. The creation of input data set based on the function's specifications
3. The determination of output based on the function's specifications
4. The execution of the test case
5. The comparison of actual and expected output
6. To check whether the application works as per the customer's need.

What is system testing ???

- **System testing** of software or hardware is **testing** conducted on a complete, integrated **system** to evaluate the **system's** compliance with its specified requirements.
- System testing is carried out by specialists testers or independent testers.
- System testing should investigate both functional and non-functional requirements of the testing.

- **Black box testing**
 - No knowledge of internal design or code required.
 - Tests are based on requirements and functionality
- **White box testing**
 - Knowledge of the internal program design and code required.
 - Tests are based on coverage of code statements, branches, paths, conditions.

BLACK BOX - TESTING TECHNIQUE

- Incorrect or missing functions
- Interface errors
- Errors in data structures or external database access
- Performance errors
- Initialization and termination errors

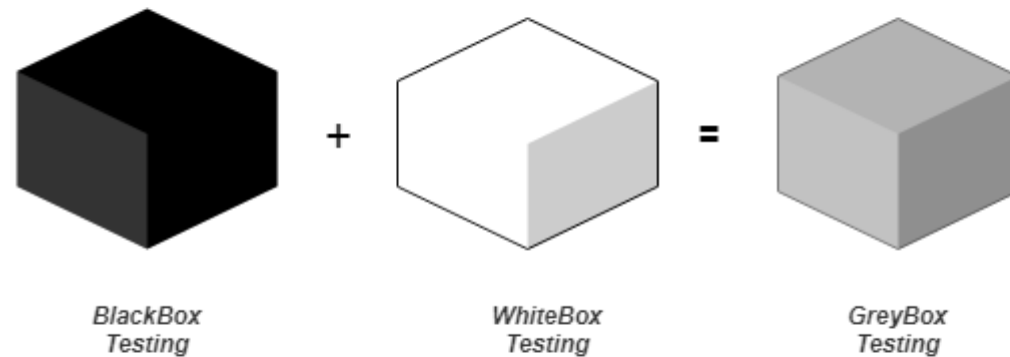
Black box / Functional testing

- Based on requirements and functionality
- Not based on any knowledge of internal design or code
- Covers all combined parts of a system
- Tests are data driven

White box testing / Structural testing

- Based on knowledge of internal logic of an application's code
- Based on coverage of code statements, branches, paths, conditions
- Tests are logic driven

Greybox testing is a software testing method to test the software application with partial knowledge of the internal working structure. It is a **combination of black box and white box testing** because it involves access to internal coding to design test cases as white box testing and testing practices are done at functionality level as black box testing.



GreyBox testing commonly identifies context-specific errors that belong to web systems. For example; while testing, if tester encounters any defect then he makes changes in code to resolve the defect and then test it again in real time. It concentrates on all the layers of any complex software system to increase testing coverage. It gives the ability to test both presentation layer as well as internal coding structure. It is primarily used in integration testing and penetration testing.

Difference between black box, white box, grey box testing.

Black-Box Testing	Grey-Box Testing	White-Box Testing
The internal workings of an application need not be known.	The tester has limited knowledge of the internal workings of the application.	Tester has full knowledge of the internal workings of the application.
Also known as closed-box testing, data-driven testing, or functional testing.	Also known as translucent testing, as the tester has limited knowledge of the insides of the application.	Also known as clear-box testing, structural testing, or code-based testing.
Performed by end-users and also by testers and developers.	Performed by end-users and also by testers and developers.	Normally done by testers and developers.

Testing is based on external expectations - Internal behavior of the application is unknown.	Testing is done on the basis of high-level database diagrams and data flow diagrams.	Internal workings are fully known and the tester can design test data accordingly.
It is exhaustive and the least time-consuming.	Partly time-consuming and exhaustive.	The most exhaustive and time-consuming type of testing.
Not suited for algorithm testing.	Not suited for algorithm testing.	Suited for algorithm testing.
This can only be done by trial-and-error method.	Data domains and internal boundaries can be tested, if known.	Data domains and internal boundaries can be better tested.